

Lecture Unit 2.1

Introduction to OpenMP

What is OpenMP?

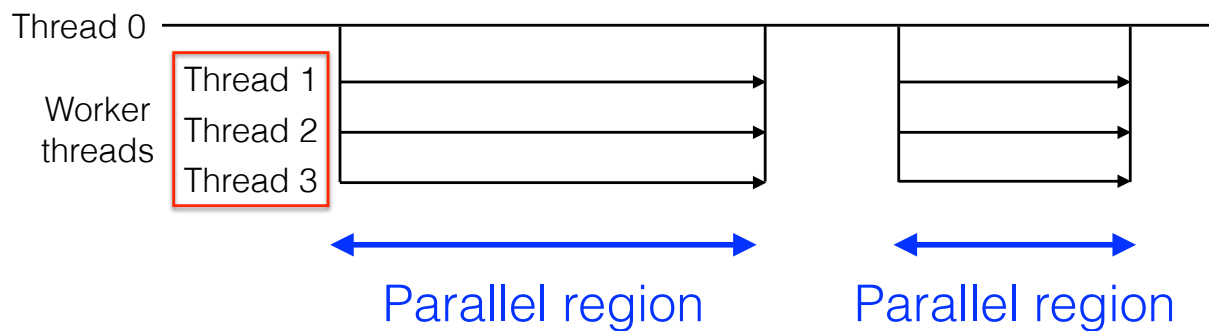
- OpenMP® (**Open Multi-Processing**) is a specification for shared memory parallel programming with C, C++ or Fortran
- OpenMP is an open standard (see <https://www.openmp.org>)
- The first standard was version 1.0 for Fortran in October 1997
- C/C++ version 1.0 was released in October 1998
- The standard continues to evolve and add new features (e.g. support for GPUs, task-based parallelism)
- **Version 6.0 is the most recent and was released in November 2024**

How do we use OpenMP?

- OpenMP provides extensions to C, C++ and Fortran
- These extensions enable the programmer to specify where parallelism should be added and how to add it
- The extensions provided by OpenMP are:
 - Compiler directives
 - Environment variables
 - Runtime library routines

OpenMP execution model

- OpenMP uses a fork-join execution model:
 - Execution starts with a single thread (master thread)
 - Worker threads start (fork) on entry to a parallel region
 - Worker threads exit (join) at the end of the parallel region



OpenMP directives (C/C++)

- Include OpenMP directives using `#pragma`

```
#pragma omp directive [clause1, ...]
```

- For example to start a parallel section use:

```
#pragma omp parallel
```

- Pragmas can be split over multiple lines by using backslash (\) to escape the newline(s)

Parallel Hello World

```
#include <stdio.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello World!\n");
    }

    return 0;
}
```

Parallel region enclosed in braces

Tells the compiler to start a parallel region

Unit Summary

- OpenMP is an open standard for parallel programming in a **shared memory** environment
- OpenMP provides compiler directives, environment variables and runtime library routines which add parallel extensions to C, C++ and Fortran
- OpenMP directives in C have the form:

```
#pragma omp directive [clause1, ...]
```